Task F2 — Five-Metric Benchmarking

Latency, p95, file size, accuracy and energy for M1, M2 and M3 | LogiEdge | FreightBridge Logistics Pvt. Ltd. | Component F, Module 6

At a glance

Item	Result
Accuracy and Critical recall	**Measured:** 100% for all three variants on the 57-window validation set and on 3,735 independent windows. The test is saturated, so it cannot rank them.
Where they differ	Under input noise, M2 and M3 raise more false alarms than M1: at noise 12σ, 4.2% (M1), 6.6% (M2), 5.4% (M3)
Latency, p95, file size, energy	**Not measured:** they need the TFLite files and your laptop. The harness is ready and checked; Section 5 gives the command.
Size (estimate)	Stored numbers: 3,212 B (M1), 1,568 B (M2), 910 B (M3); the .tflite file adds an estimated 1–3 KB of structure

1 Method

Metric	How it is measured
Mean latency (ms)	200 timed single-window inferences after 10 warm-up runs that are discarded; the harness cycles through validation windows
p95 latency (ms)	95th percentile of the same 200 timings
File size (KB)	Size of the .tflite file (1 KB = 1,000 bytes; bytes are also printed)
Accuracy (%)	The 57-window held-out validation set (stratified 20% split from Task D1), plus Critical recall; also 3,735 independent windows
Energy per inference (mJ)	E = P × t, with P = laptop TDP × CPU% ÷ 100. CPU% is psutil's system-wide reading while inference runs back to back for 2 s; t is the mean latency. Watts × ms = mJ.

I do not have the Lab 2 text, so the energy method above is my reading of "E = P × t with psutil CPU% and laptop TDP estimate". Please confirm.

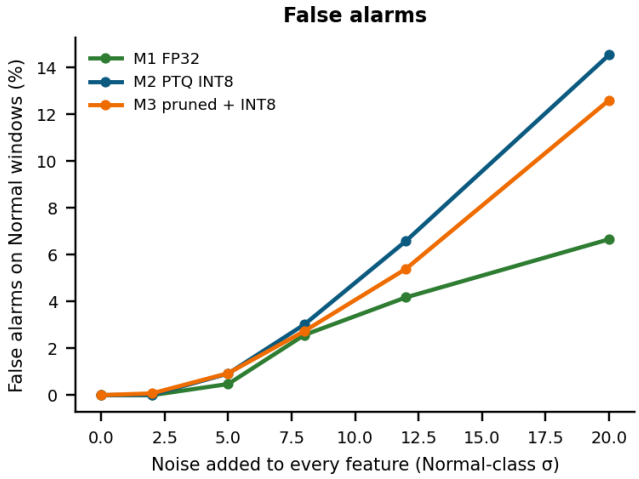
2 Results

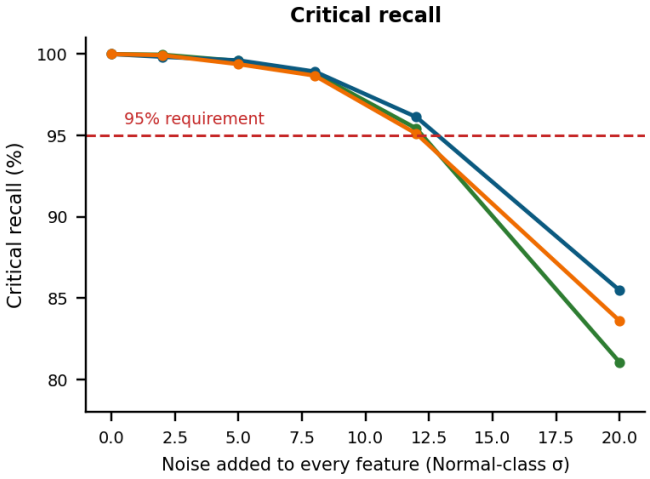
Variant	Mean latency	p95 latency	File size	Accuracy (validation)	Critical recall	Energy
M1 FP32 baseline	measure on laptop	measure on laptop	est. 4.4–5.4 KB	100.0% (57/57)	100.0% (17/17)	measure on laptop
M2 PTQ INT8	measure on laptop	measure on laptop	est. 3.2–4.4 KB	100.0% (57/57)	100.0% (17/17)	measure on laptop
M3 pruned + PTQ INT8	measure on laptop	measure on laptop	est. 2.5–3.7 KB	100.0% (57/57)	100.0% (17/17)	measure on laptop

Amber cells need a TFLite run on your laptop. Accuracy and recall come from numpy code that follows TFLite's integer arithmetic; the real .tflite files should agree within rounding. File sizes are estimates until you run `ls -l`.

Measured on independent runs	M1	M2	M3
Standard runs (1,455 windows): accuracy	100.0%	100.0%	100.0%
Runs where the fault starts mid-run (2,280 windows): accuracy	100.0%	100.0%	100.0%
Critical recall, all 3,735 windows (1,195 Critical)	100.0%	100.0%	100.0%
Agreement with M1's predictions	100%	100.0%	100.0%
Mean time from fault onset to first alert (temperature drift)	20 s	20 s	20 s
Parameters, multiply-accumulates per inference	803, 752	803, 752	400, 366
Peak activation memory	192 B	48 B	31 B

3 Where the variants differ: robustness to noise





Noise (σ)	M1 false alarms	M2	M3	M1 Critical recall	M2	M3
0	0.0%	0.0%	0.0%	100.0%	100.0%	100.0%
5	0.5%	0.9%	0.9%	99.6%	99.6%	99.4%
8	2.6%	3.0%	2.7%	98.8%	98.9%	98.7%
12	4.2%	6.6%	5.4%	95.4%	96.1%	95.1%
20	6.7%	14.5%	12.6%	81.1%	85.5%	83.6%

- Gaussian noise was added to every normalised feature of the 3,735 test windows (5 repeats). One σ is the spread of the Normal class itself.
- INT8 variants raise 1.3 to 1.6 times the false alarms of M1 at 12σ (6.6% and 5.4% against 4.2%) and about 2× at 20σ, because quantisation leaves the Normal range on one or two levels. Critical recall stays similar (M2 slightly higher, M3 slightly lower).
- Critical recall holds above 95% for every variant up to about 12σ of noise and falls to 81–86% at 20σ.

4 Harness self-check (not a result)

Variant (numpy callables)	Mean ms	p95 ms	CPU %	mJ per inference at 15 W	Accuracy
M1	0.0090	0.0153	100	0.1351	100%
M2	0.0446	0.0643	100	0.6685	100%
M3	0.0426	0.0653	100	0.6392	100%

This run only proves the harness works (200 timed runs after 10 warm-ups, p95, psutil CPU%, energy). It used numpy code on the sandbox CPU, where the INT8 emulation is slower than FP32 because it adds rounding steps; TFLite's integer kernels behave differently, so these timings say nothing about TFLite or your laptop. CPU% reads 100 because the sandbox has one core.

5 Running the real benchmark

```
python3 build_variants_tf.py
python3 benchmark.py --tflite M1=m1_fp32.tflite M2=m2_ptq_int8.tflite M3=m3_pruned_int8.tflite --tdp-watts 15
# prints the five metrics per variant and writes benchmark_results.json
```

- Use your laptop's TDP for `--tdp-watts`; 15 W is my placeholder. Close other programs so CPU% reflects the benchmark.
- These models are tiny (752 multiply-accumulates), so I expect all three to be dominated by interpreter overhead and to differ by microseconds. That is an expectation from the operation count, not a measurement.
- Send me benchmark_results.json and I will fill the amber cells and update the Task F3 reasoning.

6 Verification and items to confirm

Check	How	Result
200 timed runs after 10 discarded warm-ups; p95; energy formula; CPU% from psutil	Tests	Pass
Accuracy and Critical recall computed from per-window predictions; int8 input and output handled for .tflite files	Tests, fake interpreter	Pass
Latency, p95, file size and energy on TFLite	Needs TensorFlow and your laptop	Not run

- **Energy method** and **TDP**: confirm both (Section 1).
- **Saturated accuracy.** The simulated modes are very separable, so 100% says little about real trucks; it is the noise test that separates the variants.